PRAKTIKUM PBO 2025/2026

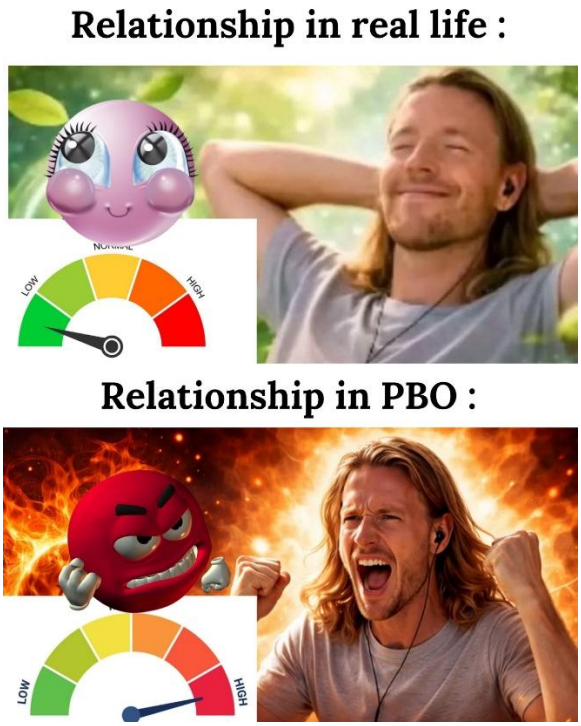
MODUL 4

RELASI

PROGRAM STUDI INFORMATIKA
UNIVERSITAS ATMA JAYA YOGYAKARTA

ERIC FULVIANO PUTRA

Relasi



Saat dengar kata *relasi*, yang langsung kepikiran biasanya soal hubungan. Di kehidupan sehari-hari, kita sering banget nemuin hubungan antar objek, misalnya antara mahasiswa dan dosen. Nah, konsep yang sama juga dipakai di Pemrograman Berorientasi Objek, di mana relasi digunakan untuk menggambarkan gimana kelas-kelas saling terhubung dan bekerja sama membentuk sebuah program yang utuh.

Tujuan Pembelajaran

Setelah mempelajari modul ini, praktikan diharapkan mampu:

1. Memahami konsep relasi antar kelas dalam Pemrograman Berorientasi Objek (PBO).
2. Mengimplementasikan berbagai jenis relasi antar kelas menggunakan bahasa pemrograman Java

Jenis-Jenis Relasi dalam PBO

A. Pewarisan (Inheritance)

Untuk relasi ini tidak akan dijelaskan lebih lanjut karena sudah dibahas di Modul Pewarisan

B. Agregasi

Agregasi merupakan hubungan **"HAS-A" (memiliki)** yang bersifat **lemah** (weak relationship). Di mana suatu objek tersusun dari objek lain atau objek penyusun masih **dapat berdiri sendiri** walaupun objek utama atau agregatnya dihapus.

Contoh:

1. **Komputer** (*Whole/Aggregate*) dengan **Printer** (*part)*

   Komputer memiliki Printer, apabila komputer dihilangkan, printer masih bisa berdiri sendiri (tetap ada).

2. **Mobil** (Whole/Aggregate) dengan **Mesin** (Part)

   Mobil memiliki mesin, apabila mobil dihilangkan, mesin masih bisa berdiri sendiri (tetap ada). **Jika objek utama dihapus, objek penyusun tetap dapat ada secara independen.**

<u>Relasi agregasi</u> ditandai dengan symbol **diamond kosong** dan multiplicity yang **bisa 0** pada sisi objek agregatnya (*whole*).

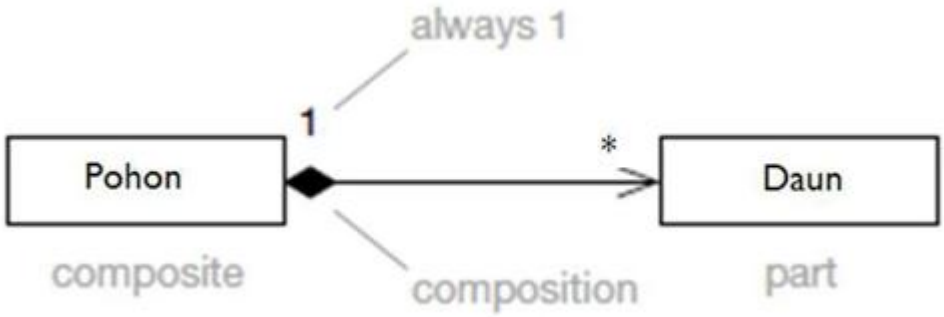
C. Komposisi

Komposisi merupakan hubungan **"HAS-A"** yang bersifat **kuat** (*strong relationship*). Dalam relasi ini objek penyusun **tidak dapat berdiri sendiri** tanpa objek utama. Jika objek utama dihapus, maka objek penyusun juga **ikut hilang** (dependent).

Contoh:

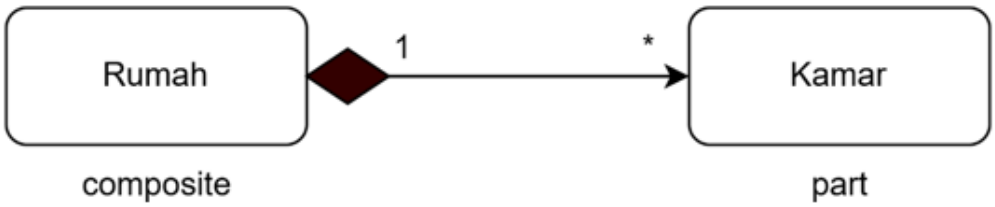
1. **Pohon** (*whale/Composite*) dengan **Daun** (*Part*)

   Pohon memiliki banyak daun, dan jika pohon ditebang, daunnya juga akan mati dan tidak bisa ada sendiri



2. **Rumah** (*whale/Composite*) dengan **Kamar** (*Part*)

   Rumah memiliki beberapa kamar, jika rumah dihancurkan, kamar juga ikut hilang karena bagian dari rumah. **Jika objek utama dihapus maka objek penyusunnya juga akan ikut hilang.**



<u>Relasi komposisi</u> ditandai dengan simbol **diamond penuh** dan multiplicity yang **harus 1** pada objek kompositnya.

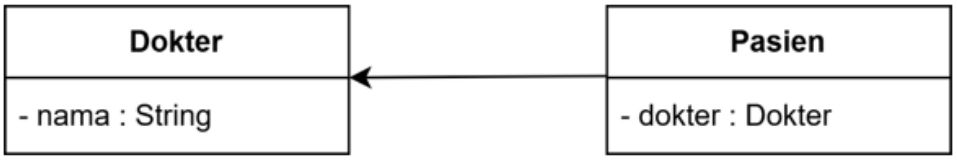
D. Asosiasi

Asosiasi dalam PBO merujuk pada **hubungan antara objek-objek** dari kelas yang berbeda, di mana satu objek dapat berinteraksi dengan objek lainnya. Konsep ini menggambarkan cara kelas dan objek saling

berhubungan satu sama lain. Asosiasi dapat dikelompokkan ke dalam beberapa jenis, seperti asosiasi **satu arah** dan asosiasi **dua arah**, serta dapat melibatkan berbagai jenis hubungan multiplicity, seperti **one-to-one, one-to-many,** atau **many-to-many.** Berikut merupakan jenis-jenis relasi asosiasi:

1. **Asosiasi Satu Arah (*Unidirectional Association*)**

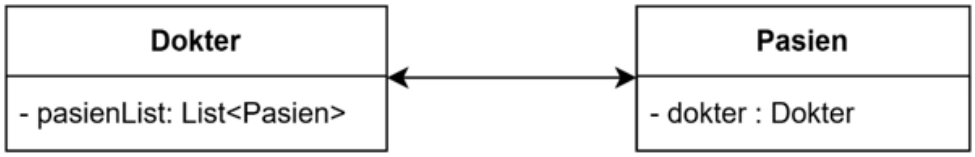
   Dalam asosiasi satu arah, satu kelas dapat mengakses objek dari kelas lain, tetapi sebaliknya tidak terjadi. Ini berarti, objek dari kelas yang satu hanya dapat merujuk atau interaksi dengan objek dari kelas lainnya tanpa sebaliknya. Contoh Asosiasi Satu Arah:



   Pada contoh di atas, kelas Pasien memiliki referensi ke objek Dokter. Namun, **Kelas Dokter tidak memiliki referensi ke objek pasien**, sehingga ini adalah asosiasi satu arah.

2. **Asosiasi Dua Arah (*Bidirectional Association*)**

   Dalam asosiasi dua arah, kedua kelas dapat **saling merujuk** atau berinteraksi satu sama lain. Contoh Asosiasi Dua Arah:



   Pada contoh ini, baik objek Dokter maupun pasien saling mengetahui satu sama lain. Jika sebuah pasien ditambahkan ke dokter, maka Dokter akan memiliki referensi ke daftar pasien yang ditangani, dan objek pasien tahun siapa dokter yang menangani mereka.

E. Dependensi

Dependensi adalah hubungan di mana **satu kelas bergantung pada kelas lain** untuk berfungsi dengan benar. Dalam konteks ini, Perubahan pada kelas yang menyediakan layanan bisa **berdampak** pada kelas yang menggunakannya. Dependensi menunjukkan hubungan ketergantungan sementara antaa suatu kelas (A) dengan kelas lain (B). Ketika kelas A mengalami perubahan, kelas B yang bergantung padanya dapat terpengaruh, tetapi hubungan ini **tidak bersifat permanen.**

Beberapa penerapan dependensi dalam kode:

- Penggunaan kelas B sebagai **parameter** dalam metode kelas A contoh : `void cetakLaporan(Laporan laporan)`
- Penggunaan kelas B sebagai **nilai balikan** dalam metode kelas A contoh : `Laporan ambilLaporanTerbaru()`
- Penggunaan kelas B sebagai **variable local** dalam metode kelas A contoh: `void prosesLaporan(){Laporan laporan = new Laporan();}`

Perbedaan Agregasi vs Komposisi vs Asosiasi

Aspek	Asosiasi	Agregasi	Komposisi
Ketergantungan	Tidak bergantung	Lemah	Kuat
Whole-Part	Tidak ada konsep "Whole-Part"	Ada, tetapi part bisa hidup sendiri	Ada, dan part tidak bisa hidup sendiri
Jika salah satu objek dihapus?	Tidak ada efek ke objek lain	Part tetap ada	Part ikut terhapus
Contoh	Dokter ↔ Pasien	Komputer → Printer	Pohon → Daun

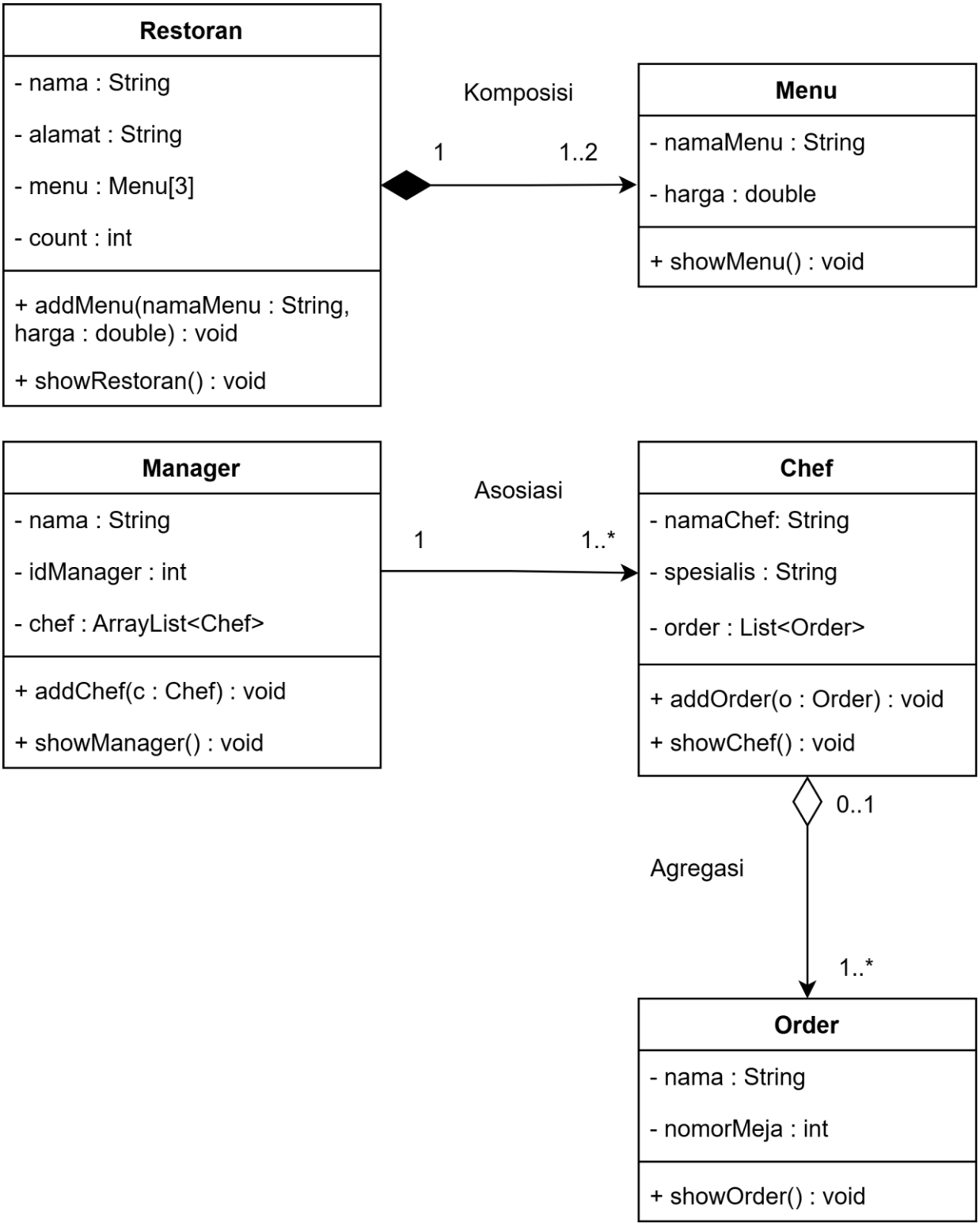
ArrayList dalam Relasi Objek

ArrayList merupakan implementasi dari *List* yang digunakan untuk menyimpan kumpulan objek secara **dinamis** (tidak dibatasi ukurannya). Dalam PBO, ArrayList sering digunakan untuk merepresentasikan hubungan ***one-to-many*** atau ***many-to-many*** antar objek.

Method yang digunakan pada ArrayList **sama** seperti yang sudah dibahas dalam modul Polimorfisme, jadi relasi ini tidak akan dijelaskan lebih lanjut karena sudah dibahas di **Modul Polimorfisme**

Guided

Pada Guided kali ini, kita akan melakukan implementasi pada konsep-konsep relasi yang sudah dijelaskan dalam modul. Berikut adalah UML diagram dan relasi-relasi yang akan diimplementasikan.



1. **Relasi Agregasi**

   Relasi agregasi terjadi antara kelas chef dan kelas Order. Di mana kelas **Chef sebagai agregat** dan kelas **Order sebegai penyusunnya.**

2. **Relasi Asosiasi**

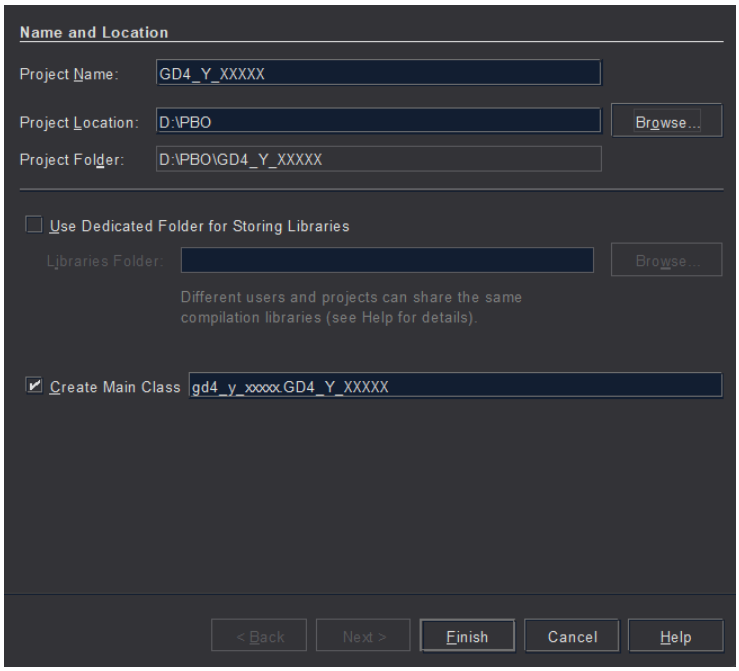
   Relasi asosiasi satu arah terjadi antara kelas Manager dengan kelas Chef. Relasi ini menunjukan bahwa seorang **Manager bisa mengelola banyak Chef**, tetapi **Chef tetap bisa berdiri sendiri tanpa Manager**.

3. **Relasi Komposisi**

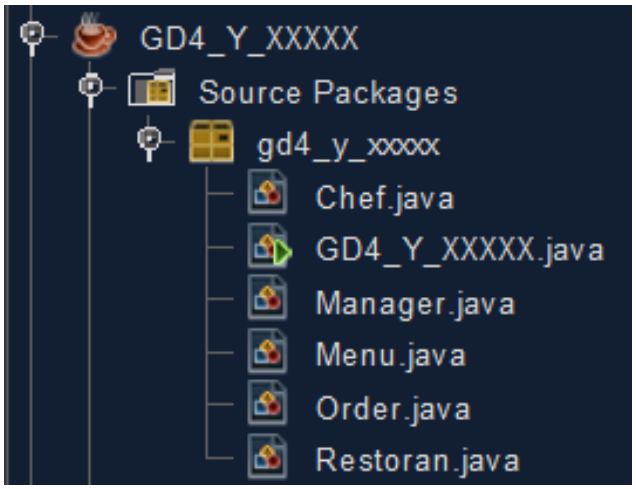
   Relasi komposisi terjadi antara kelas Restoran dan kelas Menu. Di mana **kelas Restoran menjadi komposit** dan **kelas Menu sebagai penyusunnya**.

Langkah-langkah pengerjaan :

1. Buatlah project baru pada Netbeans dengan nama **GD4_Y_XXXXX (Y = Kelas Praktikan, XXXXX = 5 digit NPM terakhir).**



2. Buatlah 5 kelas baru dengan nama **Chef, Manager, Menu, Order, Restoran**



3. Kita akan mulai dari relasi Agregasi dan Asosiasi. Silahkan isi **kelas Order** sebagai objek penyusun dari **kelas Chef** dengan kode sebagai berikut :

```
package gd4_y_xxxxx;

/**
 *
 * @NAMA LENGKAP - KELAS - NPM LENGKAP
 */
public class Order {
    private String nama;
    private int nomorMeja;

    public Order(String nama, int nomorMeja) {
        this.nama = nama;
        this.nomorMeja = nomorMeja;
    }

    public void showOrder(){
        System.out.println("\t\tNama yang pesan  : " + nama);
        System.out.println("\t\tNomor Meja       : " + nomorMeja);
    }
}
```

4. Lalu kita akan mengisi Chef sebagai objek agregat dari kelas Order.

```java
package gd4_y_xxxxx;
import java.util.ArrayList;

/**
 *
 * @author NAMA LENGKAP - KELAS - NPM LENGKAP
 */
public class Chef {
    private String namaChef;
    private String spesialis;
    private ArrayList<Order> order; // Relasi one-to-many: satu club memiliki banyak permain

    public Chef(String namaChef, String spesialis) {
        this.namaChef = namaChef;
        this.spesialis = spesialis;
        this.order = new ArrayList<>(); // pendefinisian Array List
    }

    public void addOrder(Order o){
        order.add(o);
    }

    public void showChef(){
        System.out.println("\tnama Chef      : " + namaChef);
        System.out.println("\tnama spesialis : " + spesialis);
        System.out.println("\tjumlah Order  : " + order.size());
        for(int i = 0; i<order.size(); i++){
            System.out.println("\n\t\t===== Tampil Order =====");
            System.out.println("\t\tOrder ke -  : " + (i+1));
            order.get(i).showOrder();
        }
    }
}
```

Dari kode di sini, silahkan cermati pada method **addOrder**. Tampak bahwa method ini menyimpan objek Order yang dibuat **DI LUAR** fungsi tersebut. Objek yang dibuat di luar tadi, akan di-passing di parameter **addOrder(Order o).**

Hal ini selaras dengan yang dimaksud oleh relasi Agregasi, di mana objek penyusun dari Chef yaitu Order, bersifat **mandiri** karena jika objek Chef ini dihapus pun, objek Order yang kita buat di luar fungsi addOrder tadi tidak berdampak atau ikut terhapus. Ini yang menyebabkan hubungan mereka lemah, karena independensi dari objek penyusunnya.

5. Selanjutnya kelas Manager silahkan lengkapi dengan kode sebagai berikut :

```java
package gd4_y_xxxxx;
import java.util.ArrayList;
/**
 *
 * @NAMA LENGKAP - KELAS - NPM LENGKAP
 */
public class Manager {
    private String nama;
    private int idManager;
    private ArrayList<Chef> chef;

    public Manager(String nama, int idManager) {
        this.nama = nama;
        this.idManager = idManager;
        this.chef = new ArrayList<>();
    }

    public void addChef(Chef c){
        if(c != null){ // Pengecekan null pada inputan chef
            chef.add(c); // Method untuk menambahkan objek ke list
        }
    }

    public void showManager(){
        System.out.println("\n===== Tampil chef =====");
        System.out.println("Nama        : " + nama);
        System.out.println("Id Manager  : " + idManager);
        System.out.println("Jumlah Chef : " + chef.size());
        for(int i = 0; i<chef.size(); i++){
            System.out.println("\n\t===== Tampil Chef =====");
            System.out.println("\tChef ke - " + (i+1) + " :");
            chef.get(i).showChef(); // Cara pemanggilan chef melalui Arraylist
        }

    }

}
```

Dari kode di sini, silahkan cermati pada method **addChef**. Tampak bahwa method ini menyimpan objek Chef yang dibuat **DI LUAR** fungsi tersebut. Objek yang dibuat di luar tadi, akan di-passing di parameter **addChef(Chef c).**

Jika kita perhatikan, tidak ada perbedaan sintaks dari relasi agregasi dan asosiasi:

- **Agregasi** : addOrder(Order o).
- **Asosiasi**: addClub(Chef c).

Bagian yang ditebalkan sebagai objek yang disimpan oleh masing-masing kelas tadi, sama-sama dibuat **di LUAR** fungsi yang berkaitan. Body Codenya juga tidak ada perbedaan, di mana isi array penampungnya langsung diisikan dengan objek yang ada di parameternya.

6. Relasi Agregasi dan Asosiasi telah selesai diimplementasikan. Berikutnya mari kita implementasikan relasi **Komposisi**. Berikut adalah kode untuk kelas **Menu** sebagai objek penyusun dari kelas **Restoran.**

```java
package gd4_y_xxxxx;

/**
 *
 * @NAMA LENGKAP - KELAS - NPM LENGKAP
 */
public class Menu {
    private String namaMenu;
    private double harga;

    public Menu(String namaMenu, double harga) {
        this.namaMenu = namaMenu;
        this.harga = harga;
    }

    public void showMenu(){
        System.out.println("\tNama Menu  : " + namaMenu);
        System.out.println("\tHarga menu : " + harga);
    }
}
```

7. Tambahkan kode di kelas **Restoran** sebagai **komposit** dari kelas **Menu**

```java
package tugas4_y_xxxxx;
import java.util.ArrayList;
/**
 *
 * @NAMA LENGKAP - KELAS - NPM LENGKAP
 */
public class Restoran {
    private String nama;
    private String alamat;
    private Menu menu[]; // Relasi one-to-many : satu Restoran punya banyak menu
    private int count = 0;

    public Restoran(String nama, String alamat) {
        this.nama = nama;
        this.alamat = alamat;
        this.menu = new Menu[3];
    }

    public void addMenu(String namaMenu, double harga){
        menu[count] = new Menu(namaMenu, harga);
        count++;
    }

    public void showRestoran(){
        System.out.println("===== Tampil Restoran =====");
        System.out.println("Nama Restoran   : " + nama);
        System.out.println("Alamat Restoran : " + alamat);
        System.out.println("Jumlah Menu     : " + count);
        System.out.println("\n\t===== Tampil Menu =====");
        for(int i =0; i<count; i++){
            System.out.println("\n\tData Menu ke - : " + (i+1));
            menu[i].showMenu();
        }
    }
}
```

Terdapat perbedaan pada kode relasi **komposisi**, yaitu pembuatan objek Menu dilakukan **DI DALAM** method **addMenu**. Hal ini dilakukan karena ketika **Restoran** tidak ada, maka **Menu** juga akan ikut hilang.

8. Terakhir kita lanjut ke main class kita yaitu GD4_Y_XXXXX. Silahkan isikan kode berikut untuk main class project ini:

```java
package gd4_y_xxxxx;

/**
 *
 * @NAMA LENGKAP - KELAS - NPM LENGKAP
 */
public class GD4_Y_XXXXX {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        System.out.println("\n===== GUIDED MODUL 4 RELASI PBO 25/26 =====");

        System.out.println("\n[1]. Relasi Agregasi dan Asosiasi");
        Order order1 = new Order("Kika", 4);
        Order order2 = new Order("Nevil", 13);
        Order order3 = new Order("Lucas", 8);
        Order order4 = new Order("Mukti", 15);

        Chef chef1 = new Chef("Sinay", "Sauce");
        Chef chef2 = new Chef("Mikel", "Pastry");
        Chef chef3 = new Chef("Alex", "Grill");
        Manager manager1 = new Manager("Eric", 12323);

        manager1.addChef(chef1);
        manager1.addChef(chef2);
        manager1.addChef(chef3);

        chef1.addOrder(order1);
        chef2.addOrder(order2);
        chef3.addOrder(order3);
        chef3.addOrder(order4);


        manager1.showManager();

        System.out.println("\n[2]. Relasi Komposisi");
        Restoran r1 = new Restoran("Mie Gacoan", "Jl.Babarsari no 44");
        r1.addMenu("Mie Suit", 9000);
        r1.addMenu("Mie Hompimpa", 9000);
        r1.addMenu("Udang Keju", 10000);
        r1.showRestoran();



    }

}
```

9. Output Guided, jika sudah sama maka teman-teman sudah menyelesaiakn guided modul relasi!.

```
===== GUIDED MODUL 4 RELASI PBO 25/26 =====

[1]. Relasi Agregasi dan Asosiasi

===== Tampil chef =====
Nama        : Eric
Id Manager  : 12323
Jumlah Chef : 3

        ===== Tampil Chef =====
        Chef ke - 1 :
        nama Chef      : Sinay
        nama spesialis : Sauce
        jumlah Order   : 1

                ===== Tampil Order =====
                Order ke -   : 1
                Nama yang pesan  : Kika
                Nomor Meja       : 4

        ===== Tampil Chef =====
        Chef ke - 2 :
        nama Chef      : Mikel
        nama spesialis : Pastry
        jumlah Order   : 1

                ===== Tampil Order =====
                Order ke -   : 1
                Nama yang pesan  : Nevil
                Nomor Meja       : 13

        ===== Tampil Chef =====
        Chef ke - 3 :
        nama Chef      : Alex
        nama spesialis : Grill
        jumlah Order   : 2

                ===== Tampil Order =====
                Order ke -   : 1
                Nama yang pesan  : Lucas
                Nomor Meja       : 8
```

```
                ===== Tampil Order =====
                Order ke -   : 2
                Nama yang pesan  : Mukti
                Nomor Meja       : 15

[2]. Relasi Komposisi
===== Tampil Restoran =====
Nama Restoran   : Mie Gacoan
Alamat Restoran : Jl.Babarsari no 44
Jumlah Menu     : 3

        ===== Tampil Menu =====

        Data Menu ke - : 1
        Nama Menu  : Mie Suit
        Harga menu : 9000.0

        Data Menu ke - : 2
        Nama Menu  : Mie Hompimpa
        Harga menu : 9000.0

        Data Menu ke - : 3
        Nama Menu  : Udang Keju
        Harga menu : 10000.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Format Pengumpulan :

Kumpulkan Guided dengan melakukan **zip** pada project yang telah dibuat, dengan format penamaan dibawah ini:

GD4_Y_XXXXX.zip

Y = Kelas

XXXXX = 5 digit NPM

Hint

- Silahkan pelajari bagaimana relasi ini berhubungan untuk membuat sebuah sistem, misalnya agregasi bersamaan dengan komposisi.
- Pelajari bagaimana **kardinalitas** antara relasi itu, apakah 0..1, 1..*, dan sebagainya.
- Pelajari juga bagaimana cara menggunakan **ArrayList**, serta method-method nya **(di Modul Polimorfisme)**.

Jika ada yang ingin ditanyakan, silahkan menguhubungi:

Whatsapp: 082283259667 (Eric Fulviano Putra)

Teams : 230712323@students.uajy.ac.id

Sumber Pembelajaran:

https://www.youtube.com/watch?v=KzexzfQRxDE

https://www.w3schools.com/java/java_arraylist.asp

--- Selamat Belajar ---